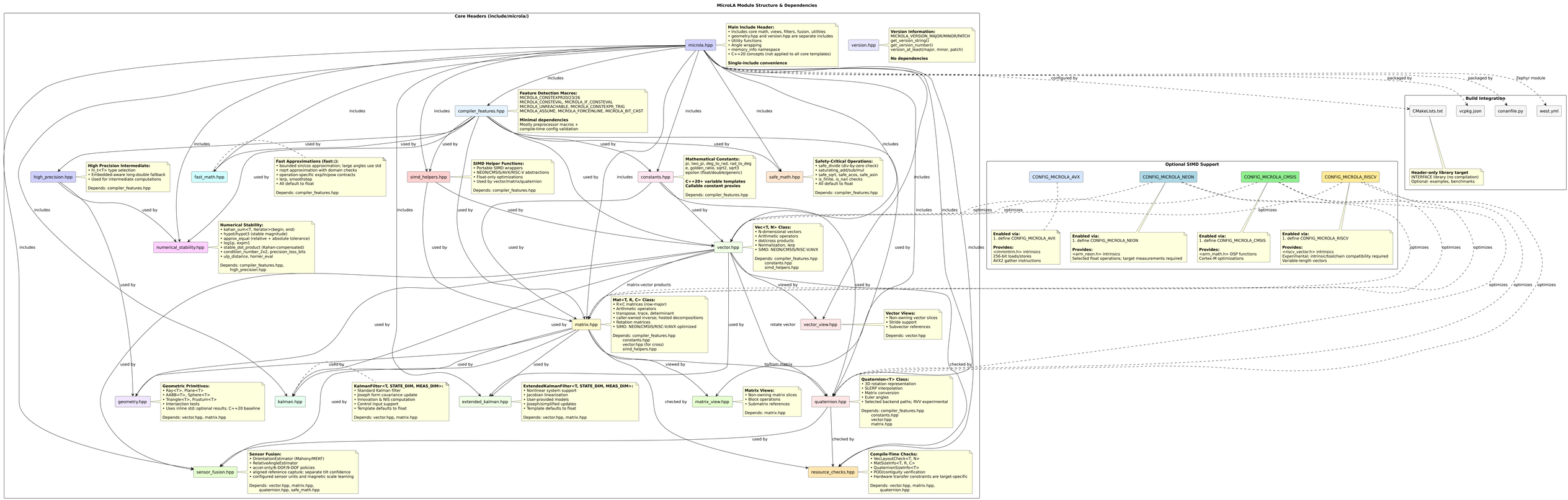
MicroLA Module Structure & Dependencies
Core Headers (include/microla/)
microla.hpp
Main Include Header:
• Includes core math, views, filters, fusion, utilities
• geometry.hpp and version.hpp are separate includes
• Utility functions
• Angle wrapping
• memory_info namespace
• C++20 concepts (not applied to all core templates)
Single-include convenience
version.hpp
Version Information:
MICROLA_VERSION_MAJOR/MINOR/PATCH
get_version_string()
get_version_number()
version_at_least(major, minor, patch)
No dependencies
compiler_features.hpp
Feature Detection Macros:
MICROLA_CONSTEXPR20/23/26
MICROLA_CONSTEVAL, MICROLA_IF_CONSTEVAL
MICROLA_UNREACHABLE, MICROLA_CONSTEXPR_TRIG
MICROLA_ASSUME, MICROLA_FORCEINLINE, MICROLA_BIT_CAST
Minimal dependencies
Mostly preprocessor macros +
compile-time config validation
high_precision.hpp
High Precision Intermediate:
• hi_txt> type selection
• Embedded-aware long-double fallback
• Used for intermediate computations
Depends: compiler_features.hpp
fast_math.hpp
Fast Approximations (fast::):
• bounded sin/cos approximation; large angles use std
• rsqrt approximation with domain checks
• operation-specific exp/ln/pow contracts
• lerp, smoothstep
• All default to float
Depends: compiler_features.hpp
simd_helpers.hpp
SIMD Helper Functions:
• Portable SIMD wrappers
• NEON/CMSIS/AVX/RISC-V abstractions
• Float-only optimizations
• Used by vector/matrix/quaternion
Depends: compiler_features.hpp
constants.hpp
Mathematical Constants:
pi, two_pi, deg_to_rad, rad_to_deg
e, golden_ratio, sqrt2, sqrt3
epsilon (float/double/generic)
C++/20+ variable templates
Callable constant proxies
Depends: compiler_features.hpp
safe_math.hpp
Safety-Critical Operations:
• safe_divide (div-by-zero check)
• saturating_add/sub/mul
• safe_sqrt, safe_acos, safe_asin
• is_finite, is_nan checks
• All default to float
Depends: compiler_features.hpp
numerical_stability.hpp
Numerical Stability:
• kahan_sum<T, Iterator>(begin, end)
• hypot3hypot3 (stable magnitude)
• approx_equal (relative + absolute tolerance)
• log1p, expm1
• stable_dot_product (Kahan-compensated)
• condition_number_2x2, precision_loss_bits
• ulp_distance, horner_eval
Depends: compiler_features.hpp,
high_precision.hpp
vector.hpp
Vec<T, N> Class:
• N-dimensional vectors
• Arithmetic operators
• dot/cross products
• Normalization, lerp
• SIMD: NEON/CMSIS/RISC-V/AVX
Depends: compiler_features.hpp
constants.hpp
simd_helpers.hpp
matrix.hpp
Mat<T, R, C> Class:
• R×C matrices (row-major)
• Arithmetic operators
• transpose, trace, determinant
• caller-owned inverse; hosted decompositions
• Rotation matrices
• SIMD: NEON/CMSIS/RISC-V/AVX optimized
Depends: compiler_features.hpp
constants.hpp
vector.hpp (for cross)
simd_helpers.hpp
vector_view.hpp
Vector Views:
• Non-owning vector slices
• Stride support
• Subvector references
Depends: vector.hpp
geometry.hpp
Geometric Primitives:
• Ray<T>, Plane<T>
• AABB<T>, Sphere<T>
• Triangle<T>, Frustum<T>
• Intersection tests
• Uses inline std::optional results; C++20 baseline
Depends: vector.hpp, matrix.hpp
kalman.hpp
KalmanFilter<T, STATE_DIM, MEAS_DIM>:
• Standard Kalman filter
• Joseph form covariance update
• Innovation & NIS computation
• Control input support
• Template defaults to float
Depends: vector.hpp, matrix.hpp
extended_kalman.hpp
ExtendedKalmanFilter<T, STATE_DIM, MEAS_DIM>:
• Nonlinear system support
• Jacobian linearization
• User-provided models
• Joseph/simplified updates
• Template defaults to float
Depends: vector.hpp, matrix.hpp
matrix_view.hpp
Matrix Views:
• Non-owning matrix slices
• Block operations
• Submatrix references
Depends: matrix.hpp
quaternion.hpp
Quaternion<T> Class:
• 3D rotation representation
• SLERP interpolation
• Matrix conversion
• Euler angles
• Selected backend paths; RVV experimental
Depends: compiler_features.hpp
constants.hpp
vector.hpp
matrix.hpp
sensor_fusion.hpp
Sensor Fusion:
• OrientationEstimator (Mahony/MEKF)
• RelativeAngleEstimator
• accel-only/6-DOF/9-DOF policies
• aligned reference capture; separate tilt confidence
• configured sensor units and magnetic scale learning
Depends: vector.hpp, matrix.hpp,
quaternion.hpp, safe_math.hpp
resource_checks.hpp
Compile-Time Checks:
• VecLayoutCheck<T, N>
• MatSizeInfo<T, R, C>
• QuaternionSizeInfo<T>
• POD/contiguity verification
• Hardware transfer constraints are target-specific
Depends: vector.hpp, matrix.hpp,
quaternion.hpp
Optional SIMD Support
CONFIG_MICROLA_AVX
Enabled via:
1. define CONFIG_MICROLA_AVX
Provides:
<immintrin.h> intrinsics
256-bit loads/stores
AVX2 gather instructions
CONFIG_MICROLA_NEON
Enabled via:
1. define CONFIG_MICROLA_NEON
Provides:
<arm_neon.h> intrinsics
Selected float operations; target measurements required
CONFIG_MICROLA_CMSIS
Enabled via:
1. define CONFIG_MICROLA_CMSIS
Provides:
<arm_math.h> DSP functions
Cortex-M optimizations
CONFIG_MICROLA_RISCV
Enabled via:
1. define CONFIG_MICROLA_RISCV
Provides:
<riscv_vector.h> intrinsics
Experimental; intrinsic/toolchain compatibility required
Variable-length vectors
Build Integration
CMakeLists.txt
vcpkg.json
conanfile.py
west.yml
Header-only library target
INTERFACE library (no compilation)
Optional: examples, benchmarks
includes
used by
configured by
packaged by
Zephyr module
optimizes
matrix-vector products
viewed by
rotate vector
to/from matrix
checked by